L.N. MISHRA COLLEGE OF BUSINESS MANAGEMENT

MUZAFFARPUR, BIHAR • AUTONOMOUS INSTITUTION

ANNUAL INTER-COLLEGIATE TECHNICAL EXTRAVAGANZA

CODTHON 2026

CODING COMPETITION

OFFICIAL SYLLABUS & COMPETITION GUIDELINES



C	C++	JAVA	PYTHON	WEB DEV	DSA
LANG	OOP	CORE	LOGIC	FULL UI	ALGORITHMS

Two-Tier Comprehensive Evaluation

Phase 1: 40 MCQs (80 Marks) • Phase 2: 2 Coding Problems (20 Marks)

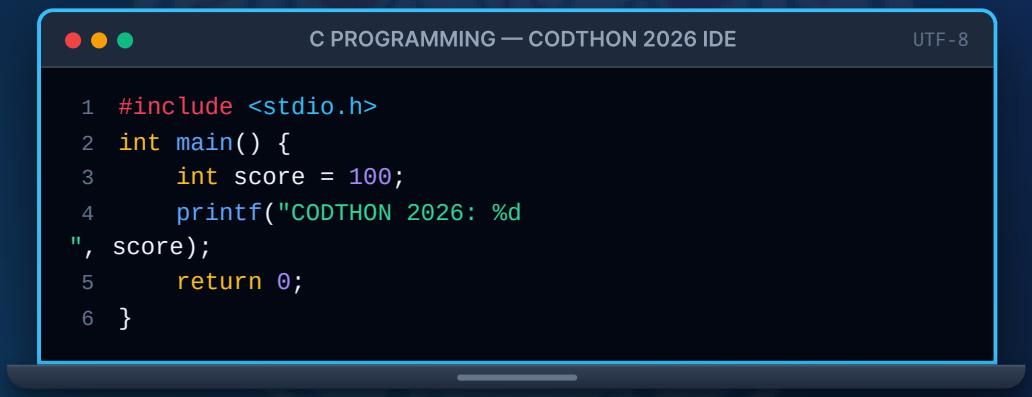
Total Score: 100 Marks

Student-Ready Curriculum & Rulebook

SECTION A

C PROGRAMMING

Programming Fundamentals, Logic & Problem-Solving



Phase 1	Phase 2	Evaluation
40 MCQs / 45 Mins	2 Coding Problems / 30 Mins	100 Maximum Marks

SECTION A — C PROGRAMMING

Programming Fundamentals, Logic & Problem-Solving • Detailed Syllabus (Units 1 to 5)

SYLLABUS SHEET 1/2

1 1. Introduction to C

Foundations

- Introduction to C language
- Header files: stdio.h
- printf() and scanf()
- Structure of a C program
- main() function
- Compilation and execution basics

2 2. Variables and Data Types

Data Representation

- Variables and constants
- Basic data types: int, char, float, double
- Basic rules for naming variables
- Variable declaration and initialization
- Format specifiers: %d, %c, %f, %lf

3 3. Operators

Expressions & Logic

- Arithmetic operators: +, -, *, /, %
- Logical operators: &&, ||, !
- Increment and decrement: ++, --
- Relational operators: >, <, >=, <=, ==, !=
- Assignment operators: =, +=, -=
- Operator precedence and associativity (basic)

4 4. Conditional Statements

Branching Logic

- if statement
- else-if ladder
- switch-case statements
- if-else statement
- Nested if constructs
- break in switch

5 5. Loops

Iteration & Flow Control

- for loop mechanics
- do-while loop execution
- break and continue statements
- Predicting the number of iterations
- while loop constructs
- Nested loop tracing
- Loop initialization, condition, and update

🎯 **PEDAGOGICAL OBJECTIVE & ASSESSMENT FOCUS (UNITS 1–5):**

Evaluates rigorous understanding of compilation phases, correct variable typing, operator short-circuiting in logical expressions, conditional branches, and manual tracing of nested loops. Tricky questions test boundary condition edge-cases.

SECTION A — C PROGRAMMING (CONTINUED)

Arrays, Strings, Modular Functions & Output Prediction • Units 6 to 10

SYLLABUS SHEET 2/2

6 6. Arrays

Linear Storage

- Introduction to arrays
- Array indexing (0-based)
- Traversing arrays using loops
- Basic array output prediction
- Declaration & initialization of 1D arrays
- Accessing and updating elements
- Finding sum, average, maximum, and minimum

7 7. Strings

Character Arrays

- Character arrays representation
- Null character (basic understanding)
- String functions: strlen(), strcpy(), strcmp(), strcat()
- String declaration and initialization
- Reading and displaying strings (basic)

8 8. Functions — Basic Level

Modularity

- Function declaration and definition
- Parameters and arguments
- Library functions vs. user-defined functions
- Function calling mechanisms
- Return values and void types

9 9. Pointers — Introduction Only

Address Basics

- Meaning of a pointer
- Address operator &
- Basic relationship between pointers and variables
- Pointer declaration
- Dereference operator *

10 10. Output Prediction and Debugging

Code Tracing

- Predicting the output of short C programs
- Identifying logical errors
- Understanding increment/decrement behavior
- Identifying syntax errors
- Tracing loops and conditions step-by-step

TOPICS EXPLICITLY EXCLUDED FROM SECTION A (C PROGRAMMING)

Dynamic memory allocation (malloc, calloc, realloc, free), complex pointer arithmetic / double pointers, linked lists, trees, graphs, advanced file handling, and advanced data structures. Questions will strictly not be set from these excluded areas.

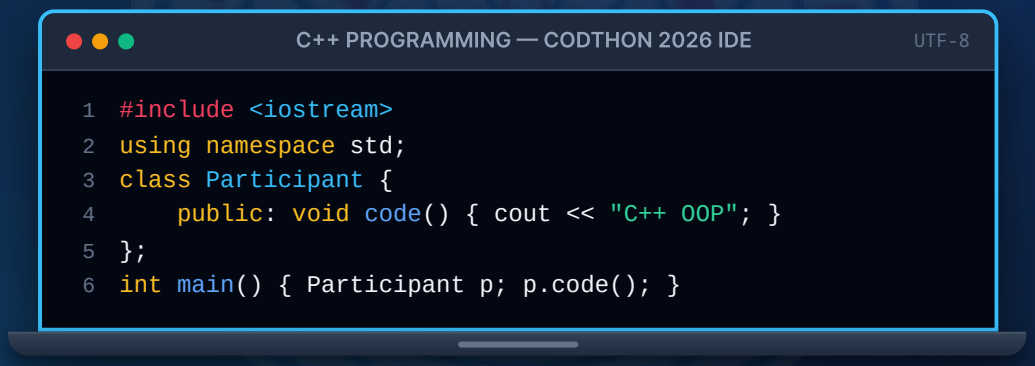
PRACTICAL CODE & TRACING FOCUS (UNITS 6–10):

Emphasis is placed on linear array traversals, null-terminated string handling, modular function parameter passing, pointer dereferencing basics, and pinpointing off-by-one errors during code debugging.

SECTION B

C++ PROGRAMMING

Logic, Object-Oriented Programming & Problem-Solving



Phase 1	Phase 2	Evaluation
40 MCQs / 45 Mins	2 Coding Problems / 30 Mins	100 Maximum Marks

SECTION B — C++ PROGRAMMING

Logic, Object-Oriented Programming & Problem-Solving • Detailed Syllabus (Units 1 to 5)

SYLLABUS SHEET 1/2

1 1. Introduction to C++

C++ Essentials

- Introduction to C++
- Header files: iostream
- cin and cout
- Basic program structure
- main() function
- using namespace std (basic understanding)

2 2. Variables and Data Types

Data & I/O

- Variables and constants
- Declaration and initialization
- Basic input and output
- int, char, float, double, bool, string
- Type conversion (basic)

3 3. Operators

Evaluation & Precedence

- Arithmetic operators
- Logical operators
- Increment and decrement operators
- Relational operators
- Assignment operators
- Operator precedence (basic)

4 4. Conditional Statements

Selection Constructs

- if
- else-if ladder
- switch-case
- if-else
- Nested if
- break and default

5 5. Loops

Repetition Control

- for
- do-while
- break and continue
- while
- Nested loops
- Loop tracing and iteration counting

🎯 **PEDAGOGICAL OBJECTIVE & ASSESSMENT FOCUS (UNITS 1–5):**

Assesses stream I/O formatting, type-safety, relational evaluations, nested decision trees, and iteration counting. Emphasis is given to understanding standard namespace scoping and correct execution ordering.

SECTION B — C++ PROGRAMMING (CONTINUED)

Arrays, Strings, Functions, OOP Fundamentals & Debugging • Units 6 to 9

SYLLABUS SHEET 2/2

6 6. Arrays and Strings

Data Collections

- One-dimensional arrays
- Traversing arrays
- Basic string creation and manipulation
- Array initialization and indexing
- Finding sum, average, maximum, and minimum
- Basic string operations and comparisons

7 7. Functions

Modular Design

- Function declaration and definition
- Parameters and arguments
- Function overloading (basic)
- Function calling
- Return values

8 8. Object-Oriented Programming (OOP)

Classes & Inheritance

- Class and object
- Access specifiers: public, private, protected
- Encapsulation (basic)
- Base class and derived class
- Data members and member functions
- Constructors (basic)
- Single inheritance

9 9. Output Prediction and Debugging

Analysis & Debugging

- Predicting the output of short C++ programs
- Identifying syntax and logical errors
- Basic debugging of class and object code
- Tracing loops and conditions
- Understanding constructors and simple inheritance

TOPICS EXPLICITLY EXCLUDED FROM SECTION B (C++ PROGRAMMING)

Advanced templates, complex operator overloading, multiple/virtual inheritance, advanced memory management, and complex STL-based problems.

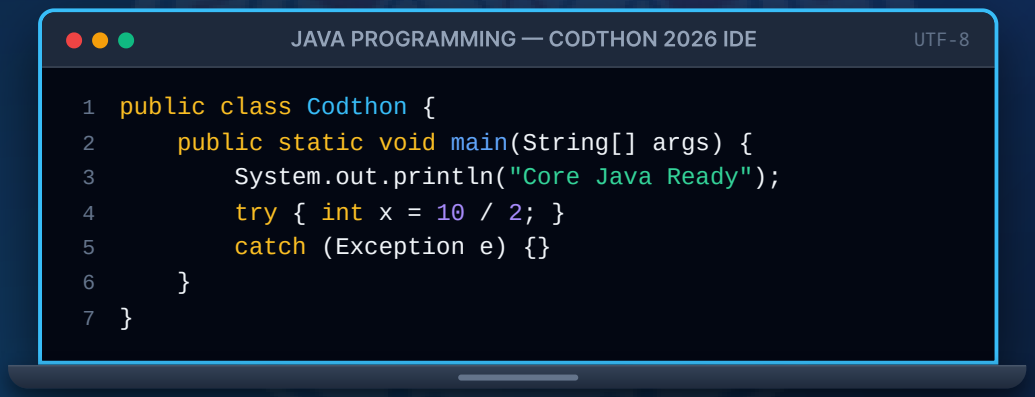
OBJECT-ORIENTED EXAM FOCUS (UNITS 6–9):

Tests constructor execution orders, access specifier enforcement, single inheritance data derivation, signature-based function overloading, and diagnosing syntax vs logical runtime issues in basic C++ classes.

SECTION C

JAVA PROGRAMMING

Core Java, Programming Logic & OOP Fundamentals



Phase 1	Phase 2	Evaluation
40 MCQs / 45 Mins	2 Coding Problems / 30 Mins	100 Maximum Marks

SECTION C — JAVA PROGRAMMING

Core Java, Programming Logic & OOP Fundamentals • Detailed Syllabus (Units 1 to 5)

SYLLABUS SHEET 1/2

1 1. Introduction to Java

JVM Architecture

- Introduction to Java
- class and main() method
- System.out.print() and System.out.println()
- Basic Java program structure
- public, static, and void (basic understanding)
- JVM, JDK, and JRE (basic concepts)

2 2. Variables and Data Types

Type System

- Variables and constants
- String
- Type casting and conversion (basic)
- Primitive data types: int, char, float, double, boolean
- Declaration and initialization
- Input/output basics

3 3. Operators

Operators & Evaluation

- Arithmetic operators
- Logical operators
- Increment and decrement
- Relational operators
- Assignment operators
- Operator precedence (basic)

4 4. Conditional Statements

Control Flow

- if
- else-if ladder
- switch-case
- if-else
- Nested if
- break and default

5 5. Loops

Iterative Execution

- for
- do-while
- break and continue
- while
- Nested loops
- Loop tracing and iteration counting

🎯 **CORE JAVA FOUNDATIONS FOCUS (UNITS 1–5):**
Focuses on Java bytecode execution fundamentals, primitive type sizes and narrowing/widening conversions, operator associativity, and loop iteration bounds across for, while, and do-while structures.

SECTION C — JAVA PROGRAMMING (CONTINUED)

Arrays, Strings, Methods, OOP, Exception Basics & Debugging • Units 6 to 10

SYLLABUS SHEET 2/2

6 6. Arrays and Strings

Linear Collections

- Array declaration and initialization
- Finding sum, average, maximum, and minimum
- length() and charAt()
- Array indexing and traversal
- String creation and basic operations
- Basic string comparison using equals()

7 7. Methods

Modular Functions

- Method declaration and calling
- Return types
- Basic static methods
- Parameters and arguments
- Methods with and without return values

8 8. Object-Oriented Programming (OOP)

Core Java OOP

- Classes and objects
- Constructors
- Encapsulation
- Method overloading (basic)
- Fields and methods
- Access modifiers (basic)
- Inheritance (basic)

9 9. Exception Handling — Basic

Error Resilience

- Meaning of an exception
- Common examples such as division by zero
- Basic try and catch
- Understanding simple exception-related code

10 10. Output Prediction and Debugging

Code Tracing

- Predicting program output
- Identifying syntax and logical errors
- Understanding simple class and method behavior
- Tracing loops and conditions
- Basic array and string debugging

TOPICS EXPLICITLY EXCLUDED FROM SECTION C (JAVA PROGRAMMING)

Multithreading, advanced generics, streams, complex collections, and advanced JVM internals.

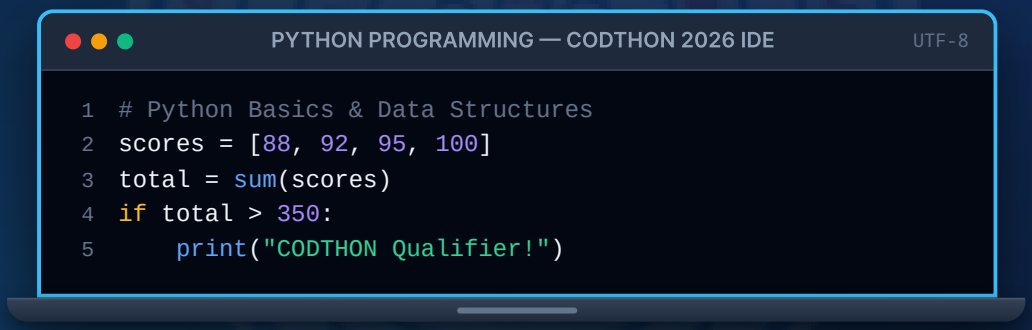
OOP & EXCEPTION HANDLING FOCUS (UNITS 6–10):

Evaluates string immutability, equals() vs == comparisons, constructor overloading, static vs instance scope, basic arithmetic exception handling, and tracing class-level output.

SECTION D

PYTHON PROGRAMMING

Python Basics, Data Types & Logical Thinking



Phase 1
40 MCQs / 45 Mins

Phase 2
2 Coding Problems / 30 Mins

Evaluation
100 Maximum Marks

SECTION D — PYTHON PROGRAMMING

Python Basics, Data Types & Logical Thinking • Detailed Syllabus (Units 1 to 6)

SYLLABUS SHEET 1/2

⚠ **IMPORTANT RESTRICTION:** Do not include user-defined functions, recursion, or exception handling in the Python syllabus or any sample question. These topics have not yet been covered by the students.

1 1. Python Basics

Syntax Fundamentals

- Introduction to Python
- Comments
- Basic program execution
- Python syntax and indentation
- print() and input()

2 2. Variables and Data Types

Dynamic Typing

- Variables and assignment
- Type checking using type()
- Multiple variable assignment (basic)
- int, float, str, bool
- Basic rules for naming variables

3 3. Type Conversion

Casting

- int()
- str()
- Basic implicit and explicit type conversion
- float()
- Converting input into numbers

4 4. Operators

Arithmetic & Logic

- Arithmetic operators: +, -, *, /, //, %, **
- Logical operators: and, or, not
- Operator precedence (basic)
- Comparison operators
- Assignment operators

5 5. Conditional Statements

Branching

- if
- if-elif-else
- Conditions using comparison and logical operators
- if-else
- Nested conditions

6 6. Loops

Iterative Loops

- for loop
- range()
- break and continue
- while loop
- Nested loops
- Loop tracing and iteration counting

🎯 **PYTHON CORE LOGIC FOCUS (UNITS 1–6):**
Rigorously evaluates indentation correctness, integer division (//) vs float division (/), exponentiation (**), type conversion of user input, logical operator short-circuiting, and range() boundary parameters.

SECTION D — PYTHON PROGRAMMING (CONTINUED)

Data Structures, Built-ins & Output Prediction • Units 7 to 13

SYLLABUS SHEET 2/2

7 7. Strings

- Creating strings
- String indexing
- String slicing
- String concatenation
- Basic methods: upper(), lower(), strip(), replace()
- len() with strings
- Predicting string output

8 8. Lists

- Creating lists
- Indexing and slicing
- Updating list elements
- append() and remove()
- len() with lists
- Traversing lists using loops
- Finding sum, max, min using built-in functions

9 9. Tuples

- Creating tuples
- Accessing elements
- Indexing
- Tuple immutability
- Difference: lists vs tuples

10 10. Dictionaries

- Creating dictionaries
- Keys and values
- Accessing via keys
- Adding/updating entries
- keys() and values() (basic)
- Checking key existence

11 11. Sets

- Creating sets
- Unique elements
- Adding/removing elements
- Union and intersection
- Difference: lists vs sets

12 12. Built-in Functions

Built-in Library

`print()` `input()` `len()` `type()` `sum()` `max()` `min()` `range()`

13 13. Output Prediction and Debugging

Logical Tracing

- Predicting the output of short Python snippets
- Tracing conditions and loops
- Understanding indexing and slicing
- Understanding indentation
- Identifying incorrect conditions
- Basic list, tuple, and dictionary errors

TOPICS EXPLICITLY EXCLUDED FROM SECTION D (PYTHON PROGRAMMING)

User-defined functions (def), recursion, exception handling (try, except, finally), classes, decorators, generators, file handling, and advanced libraries.

COLLECTIONS & DATA STRUCTURES FOCUS (UNITS 7–13):

Highlights string slicing notation [start:stop:step], list mutation methods, tuple immutability properties, dictionary hashing and key retrieval, set mathematical uniqueness, and evaluating built-ins.

SECTION E

WEB DEVELOPMENT

HTML, CSS & JavaScript Fundamentals



Phase 1	Phase 2	Evaluation
40 MCQs / 45 Mins	2 Coding Problems / 30 Mins	100 Maximum Marks

SECTION E — WEB DEVELOPMENT

HTML, CSS & JavaScript Fundamentals • Detailed Syllabus (Units 1 to 4)

SYLLABUS SHEET 1/2

1 1. HTML Fundamentals

Structure & Markup

- Structure of an HTML document
- Headings and paragraphs
- Ordered and unordered lists
- Forms and input fields
- Semantic tags: header, nav, main, section, footer
- html, head, title, and body
- Links and images
- Tables and basic table elements
- Buttons, labels, and placeholders

2 2. CSS Fundamentals

Styling & Box Model

- Inline, internal, and external CSS
- Colors and backgrounds
- Borders and border radius
- CSS box model
- Basic display properties
- Selectors: element, class, and ID
- Fonts and text styling
- Margin and padding
- Width and height

3 3. CSS Layout

Flexbox & Positioning

- Block and inline elements
- Flex direction and alignment
- Positioning: static, relative, absolute, fixed
- Media queries (basic)
- display: flex
- justify-content and align-items
- Basic responsive design

4 4. JavaScript Fundamentals

Client-Side Scripting

- Variables: let, const, var (basic differences)
- Arithmetic and comparison operators
- Type conversion (basic)
- Basic output prediction
- Data types
- Logical operators
- console.log()

🎯 **FRONTEND ARCHITECTURE FOCUS (UNITS 1–4):**
Tests semantic markup structure, selector specificity, box model dimension calculations, flexbox layout axis orientation, responsive viewport breakpoints, and core JavaScript primitive types.

SECTION E — WEB DEVELOPMENT (CONTINUED)

Control Flow, DOM Manipulation, Events & Practical Evaluation Note • Units 5 to 8

SYLLABUS SHEET 2/2

5 5. Conditional Statements and Loops

JS Control Flow

- if-else
- else-if
- switch
- for loop
- while loop
- Basic loop tracing

6 6. Functions — Basic

Functions

- Function declaration
- Function calling
- Parameters and return values
- Basic event-handler functions

7 7. DOM and Events

DOM Manipulation

- Meaning of the DOM
- Selecting elements using getElementById()
- Basic querySelector()
- Changing text using textContent or innerHTML
- Changing styles through JavaScript
- Click events
- Basic form interaction

8 8. Debugging and Practical Concepts

Integration

- Identifying HTML structure errors
- Identifying incorrect CSS selectors
- Basic JavaScript syntax errors
- Predicting the output of simple JavaScript
- Understanding why a button or form may not work
- Basic HTML/CSS/JavaScript integration

💡 **PRACTICAL EVALUATION NOTE:** Web Development coding tasks may require students to build a small functional webpage using HTML, CSS, and JavaScript. Evaluation should consider required elements, layout, responsiveness, and functionality—not just text output.

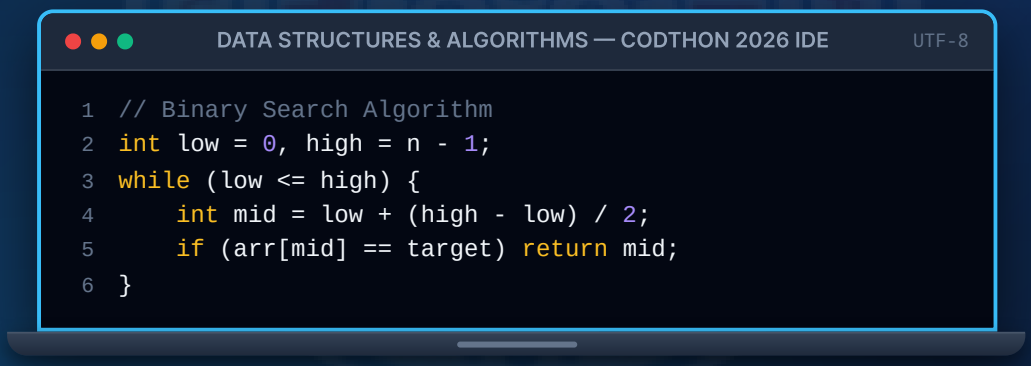
TOPICS EXPLICITLY EXCLUDED FROM SECTION E (WEB DEVELOPMENT)

React, Next.js, TypeScript, Node.js, APIs, databases, authentication, and advanced full-stack development.

SECTION F

DATA STRUCTURES & ALGORITHMS

DSA • Problem-Solving, Algorithms & Logical Thinking



Phase 1
40 MCQs / 45 Mins

Phase 2
2 Coding Problems / 30 Mins

Evaluation
100 Maximum Marks

SECTION F — DATA STRUCTURES & ALGORITHMS (DSA)

Problem-Solving, Algorithms & Logical Thinking • Detailed Syllabus (Units 1 to 5)

SYLLABUS SHEET 1/2

ℹ️ **IMPORTANT RESTRICTION:** Students may select C, C++, or Java for the DSA category. The core challenge must assess DSA fundamentals rather than advanced language features.

1 1. Algorithm Basics

Complexity Basics

- Meaning of an algorithm
- Pseudocode (basic)
- Basic time-complexity concepts
- Step-by-step problem-solving
- Input, processing, and output
- Understanding O(1), O(n), and $O(n^2)$ through simple examples

2 2. Arrays

Array Manipulation

- Array creation and initialization
- Finding maximum and minimum
- Counting elements based on a condition
- Indexing and traversal
- Finding sum and average
- Reversing an array (basic)

3 3. Strings

String Algorithms

- String traversal
- Reversing a string
- Counting vowels and consonants
- Counting characters
- Checking a palindrome
- Basic character comparisons

4 4. Searching

Search Techniques

- Linear search
- Difference between linear and binary search
- Counting search iterations (basic)
- Basic binary search
- Conditions required for binary search

5 5. Sorting

Elementary Sorting

- Bubble sort
- Ascending and descending order
- Identifying the output after one or more passes
- Selection sort
- Tracing sorting steps

🎯 **ALGORITHMIC COMPETENCIES FOCUS (UNITS 1–5):**
Evaluates algorithmic efficiency awareness (O(1) constant vs O(n) linear vs $O(n^2)$ quadratic), two-pointer array/string reversals, palindrome logic, binary search invariants, and manual sorting pass tracking.

SECTION F — DATA STRUCTURES & ALGORITHMS (CONTINUED)

Mathematics, Logic, Recursion, Stacks/Queues & Debugging • Units 6 to 10

SYLLABUS SHEET 2/2

6 6. Basic Mathematics and Logic

Numerical Logic

- Even/odd checks
- Factorial using loops
- Reversing a number
- Basic divisibility problems
- Prime number checks
- Sum of digits
- Palindrome number

7 7. Frequency Counting and Pair Problems

Hash & Pairs

- Counting occurrences of an element
- Frequency arrays (basic)
- Counting elements that satisfy a condition
- Finding duplicate elements (basic)
- Finding a pair with a given sum using simple approaches

8 8. Recursion — Basic

Recursive Flow

- Meaning of recursion
- Recursive case
- Basic factorial or sum examples
- Base case
- Tracing simple recursive calls

9 9. Stack and Queue — Fundamentals

Linear ADTs

- Meaning of stack and queue
- Push and pop concepts
- Simple operation-tracing questions
- LIFO and FIFO
- Enqueue and dequeue concepts

10 10. Output Prediction and Debugging

Algorithmic Tracing

- Tracing algorithms manually
- Identifying off-by-one errors
- Tracing basic search and sorting algorithms
- Finding logical errors in loops
- Predicting array and string outputs

TOPICS EXPLICITLY EXCLUDED FROM SECTION F (DSA)

Linked lists, trees, graphs, dynamic programming, segment trees, tries, advanced greedy algorithms, and complex backtracking.

DISCRETE PROBLEM-SOLVING & ADT FOCUS (UNITS 6–10):

Tests prime checking optimization, digit summation algorithms, frequency count mapping, two-sum target searching, recursive call stack unwinding, and stack/queue pointer transitions.

SECTION 5

COMPETITION GUIDELINES

Test Pattern, Marking Scheme & Student Requirements

⏱️
Timed Tests
45 Min MCQ + 30 Min Code

📝
Phase 1: MCQs
40 Questions • 80 Marks

💻
Phase 2: Coding
2 Problems • 20 Marks

🔒
Hidden Test Cases
5 Hidden Cases / Problem

📊
Final Scorecard
Total 100 Marks

🏆
Fair Evaluation
Strict Academic Integrity

PHASE 1 (MCQ) & PHASE 2 (CODING) TEST SPECIFICATIONS

GUIDELINES SHEET 1/2

Examination Architecture, Distributions, Test Cases & Evaluation Logic

1 Phase 1 — MCQ Test Overview (45 Mins • 80 Marks)

80 Marks Total

40	45 Min	80	Random
TOTAL MCQS	DURATION	MAXIMUM MARKS	QUESTION ORDER

- Questions must be based on the selected category's prescribed syllabus.
- Question and option order may be randomized for each participant.
- The system must preserve the correct answer mapping after randomization.
- Phase 1 scores must not be disclosed before the final scorecard.

Question-Type Distribution

Question Type	Number of Questions	Marks per Question	Total Marks
Fundamentals	10	1	10
Technical Understanding	10	2	20
Logic and Debugging	10	3	30
Case Study	10	2	20
TOTAL	**40**	**—**	**80**

Difficulty Distribution

Difficulty Level	Percentage	Number of Questions	Marks
Easy (Basic concepts, direct applications, definitions, simple output prediction)	60%	24	48
Moderate (Concept application, multi-step reasoning, basic debugging)	30%	12	24
Tricky (Careful reading, logical traps, edge cases, output prediction within syllabus)	10%	4	8
TOTAL	**100%**	**40**	**80**

** Tricky does not mean advanced or out-of-syllabus.*

2 Phase 2 — Coding Test Overview (30 Mins • 20 Marks)

Hands-On Programming

Problem 1: Easy (10 Marks)
- 5 Hidden Test Cases (2 Marks per passed test case)
- Tests fundamental problem solving and basic algorithms.

Problem 2: Easy–Moderate (10 Marks)
- 5 Hidden Test Cases (2 Marks per passed test case)
- Tests multi-step logic and boundary edge-cases.

- Total hidden test cases: 10 across both problems. Hidden test cases must not be shown to students.
- Students may run their code within the configured run limits. Each problem permits one final submission.
- A confirmation dialog should appear before final submission.
- The timer must start only after the student selects a category/language and clicks "Start Coding Test". Timer pausing or restarting is not permitted.
- If time expires before submission, the competition administrator's published policy will determine whether the latest saved code is evaluated or the problem receives zero marks.
- Web Development tasks must be evaluated for functionality and page behavior rather than only standard input/output.

TOTAL MARKS & FINAL SCORECARD ARCHITECTURE

Overall Scoring Scheme, Result Publication Standards & Certified Credential Fields

GUIDELINES SHEET 2/2

1 Overall Marking Scheme

Maximum Score: 100

Component	Maximum Marks
Phase 1 MCQ Test	80
Coding Problem 1	10
Coding Problem 2	10
TOTAL	**100**

2 Final Scorecard Specification & Display Fields

Digital Scorecard

Explain that the final scorecard is available only after both phases are complete and the result has been finalized or published. Do not invent a passing percentage, cutoff, rank, prize, or result declaration date.

The final scorecard may display:

- Participant name
- Roll number
- Course
- Email
- Participant ID
- Phase 1 marks out of 80
- Coding Problem 1 marks out of 10
- Coding Problem 2 marks out of 10
- Total marks out of 100
- Result status
- Result timestamp
- Verification ID

3 Verification & Credential Authentication

Integrity Standards

- Each final scorecard includes a unique cryptographic Verification ID to prevent alteration and permit rapid institutional verification.
- Individual question responses and test-case execution traces remain archived in the official competition logs for committee review.
- Certified scorecards will be downloadable directly from the authenticated candidate portal following formal administrative announcement.

📋 **OFFICIAL RESULT PUBLICATION POLICY:** Official scorecards will be published exclusively through the competition administration portal after comprehensive review. Cutoffs, merit positions, or declaration schedules must not be inferred prior to formal administrative release.

BEFORE YOU PARTICIPATE — MANDATORY STUDENT REQUIREMENTS

CHECKLIST

Operational Checklists, Technical Protocols & Academic Integrity Regulations

1 1. Laptop Requirement

- Every participant must have access to a working laptop for the competition.
- Participants are encouraged to bring their own laptop.
- Students who do not own a laptop may borrow one from a friend, classmate, or another permitted source before the test.
- Borrowed laptops must be arranged in advance.
- Participants must not depend on finding an available laptop after the test has started.
- The laptop must have a working keyboard, functional screen, and sufficient battery or access to power.

2 2. Internet Connectivity

- Ensure that a stable internet connection is available.
- Test the browser and internet connection before the competition begins.
- Keep mobile hotspot access available as a backup if permitted.
- Do not assume that campus Wi-Fi will be available to every participant.

3 3. Browser and Device Preparation

- Use an up-to-date supported browser.
- Close unnecessary applications and browser tabs.
- Disable disruptive notifications.
- Ensure that the device is sufficiently charged.
- Bring the laptop charger.
- Check that the selected programming language and development environment are available if required.

4 4. Login Credentials

- Participants will receive their assigned User ID and temporary password through official distribution.
- User ID will be based on the participant's roll number.
- Keep credentials private.
- Do not share credentials with another participant.
- Change the temporary password if the competition system supports and requires it.

5 5. Student Information

- Keep your college ID card available for identity verification if requested.
- Use the correct name, roll number, course, and email address.
- Report any incorrect account information to the authorized coordinator before the test.

6 6. During the Competition

- Reach the venue before the announced start time.
- Follow the invigilator's instructions.
- Do not communicate answers to other participants.
- Do not use unauthorized notes, messaging apps, websites, AI tools, or external assistance.
- Do not switch tabs or exit fullscreen if the competition rules prohibit it.
- Follow the configured monitoring and warning policy.
- Submit answers before the timer expires. Do not assume unsaved code will be accepted.

7 7. Technical Issues

- Inform the invigilator immediately if a technical issue occurs.
- Do not refresh, close, or reopen the test without authorization.
- Follow the official recovery instructions.
- Preserve any displayed error message or relevant screenshot if requested by the invigilator.

8 8. Academic Integrity

- Every participant must complete the test independently.
- Plagiarism, impersonation, sharing code, and unauthorized assistance are prohibited.
- Violations may lead to warnings, review, or disqualification.
- Important note: Monitoring and fullscreen controls may detect certain browser events but cannot guarantee that every form of unauthorized

SECTION 7 — IMPORTANT REMINDERS

PREPARE. PRACTICE. PERFORM.

"Understand your fundamentals, practice logical thinking, and compete with confidence."



✔ COMPACT FINAL CHECKLIST

- [1] Review the complete syllabus for your selected category.
- [2] Practice output prediction and basic debugging.
- [3] Keep your laptop, charger, and internet ready.
- [4] Keep your login credentials secure.
- [5] Read the competition instructions carefully.
- [6] Follow academic integrity rules.